

KING MONGKUT'S INSTITUTE OF TECHNOLOGY LATKRABANG

SCHOOL OF ENGINEERING

ROBOTIC & AI ENGINEERING

GROUP OF (RAI 1, RAI 2, HUACHIEW)



ROBOTICS COMPETITION REPORT

INSTRUCTED BY: DR. POOM KONGHUAYROB

NAME: JAIJAI, NUUN, TITLE

GROUP NO: 03

DATE OF SUB.: 10/11/2568

PURPOSE

- To practice coding for controlling real hardware such as motors, servos, and sensors.
- To create a function robots in both autonomous and manual control systems
- To develop 3D design and programming skill
- To apply physics principles to development and improvement of robotics systems for future innovation.

AUTONOMOUS ROBOT

COMPONENT LIST

- Arduino UNO

Parameter	Specification
microcontroller	ATmega328P
USB Controller	ATmega16U2
Operating Voltage	5 V
Input Voltage	7 - 12 V
Digital I/O Pins	14 (6 PWM)
Communication	UART, I ² C, SPI
Dimensions	68.6 x 53.4 mm
Weight	25 g

- TT DC Motor Dual-Shaft

Parameter	Specification
Gear Reduction Ratio	1:48
Rated Voltage	3 – 6 V
No-load Speed	≈200 RPM at 6V
No-load Current	≈150 mA at 6V
Stall Torque	≈0.8 kg cm at 6V
Stall Current	≈700 mA
Shaft Type	Double D-shaped output shaft
Gearbox Material	Plastic
Dimensions	≈70 × 22 × 18 mm
Weight	≈30 – 50 g
Power	≈0.9 W

- SG90 Servo Motor 9g

Parameter	Specification
Operating Voltage	4.8 - 6 V DC
Operating Current (No Load)	~100 mA
Stall Current	~680 mA
Torque	1.8 kg cm at 4.8 V
Speed	0.1 s/60° at 4.8 V
Rotation Angle	~180°
Weight	9 g

- Gyro Sensor MPU6050 GY-521

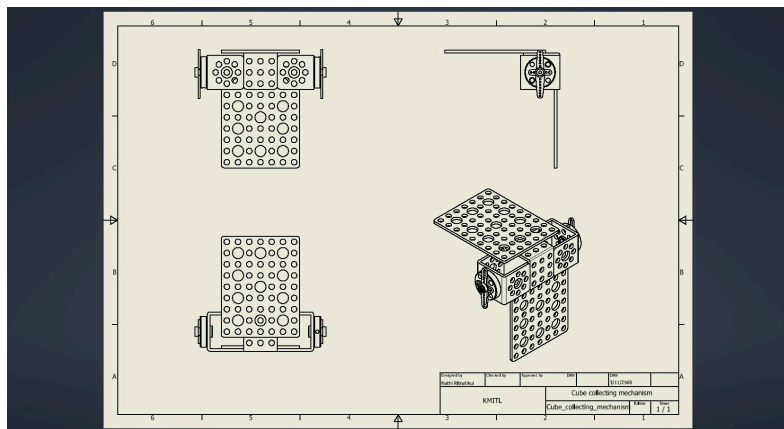
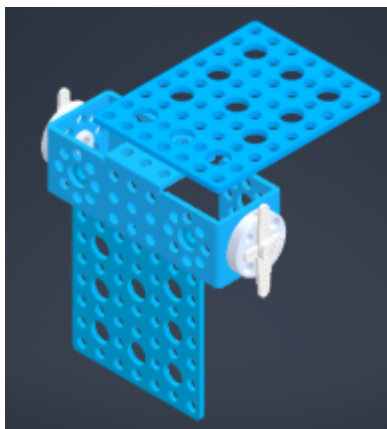
Parameter	Specification
Supply Voltage	3 - 5 V DC
Gyro Range	± 250 , ± 500 , ± 1000 , ± 2000 °/s
Accelerometer Range	$\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$
Operating Current	~ 3.9 mA
Communication Speed	400 kHz I ² C

- L298N

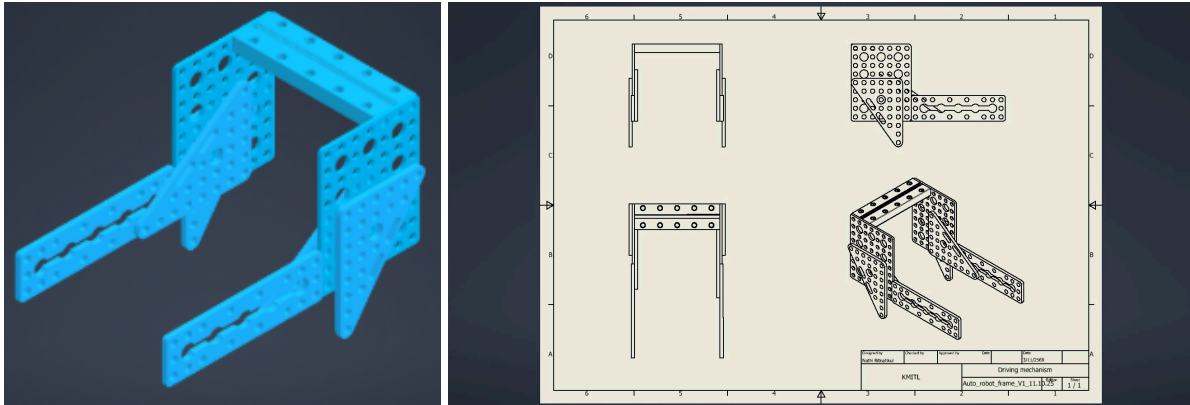
Parameter	Specification
Motor Supply Voltage	5 - 35 V DC
Logic Supply Voltage	5 V
Continuous Output Current	2 A per channel
Channels	2 DC motors
Control Inputs	IN1, IN2, IN3, IN4
PWM Control	ENA, ENB
Peak Output Current	3 A per channel
Thermal Shutdown	Build in

ASSEMBLY DESIGN

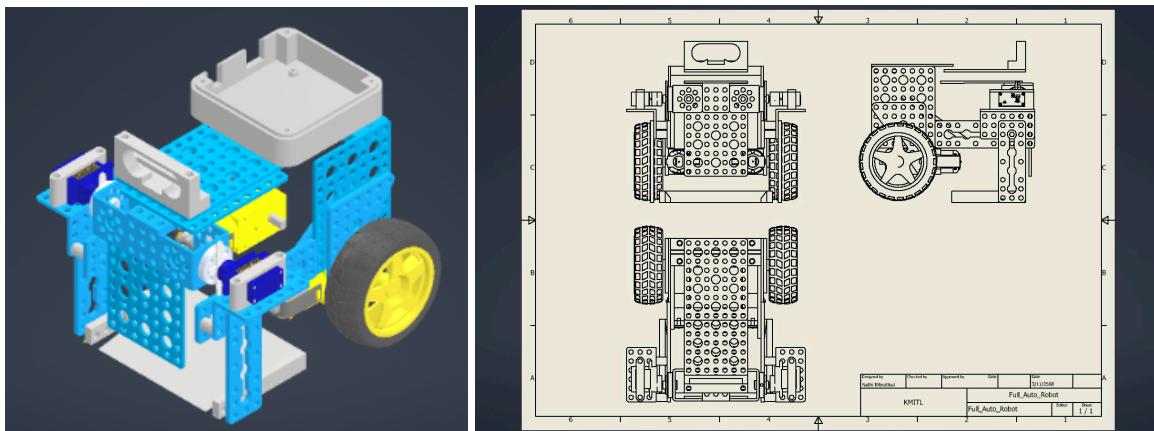
1.1 Cube Collecting Mechanism



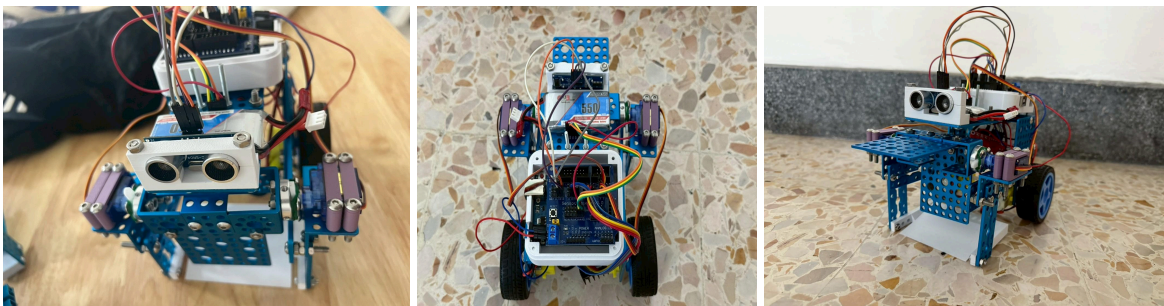
1.2 Driving Mechanism



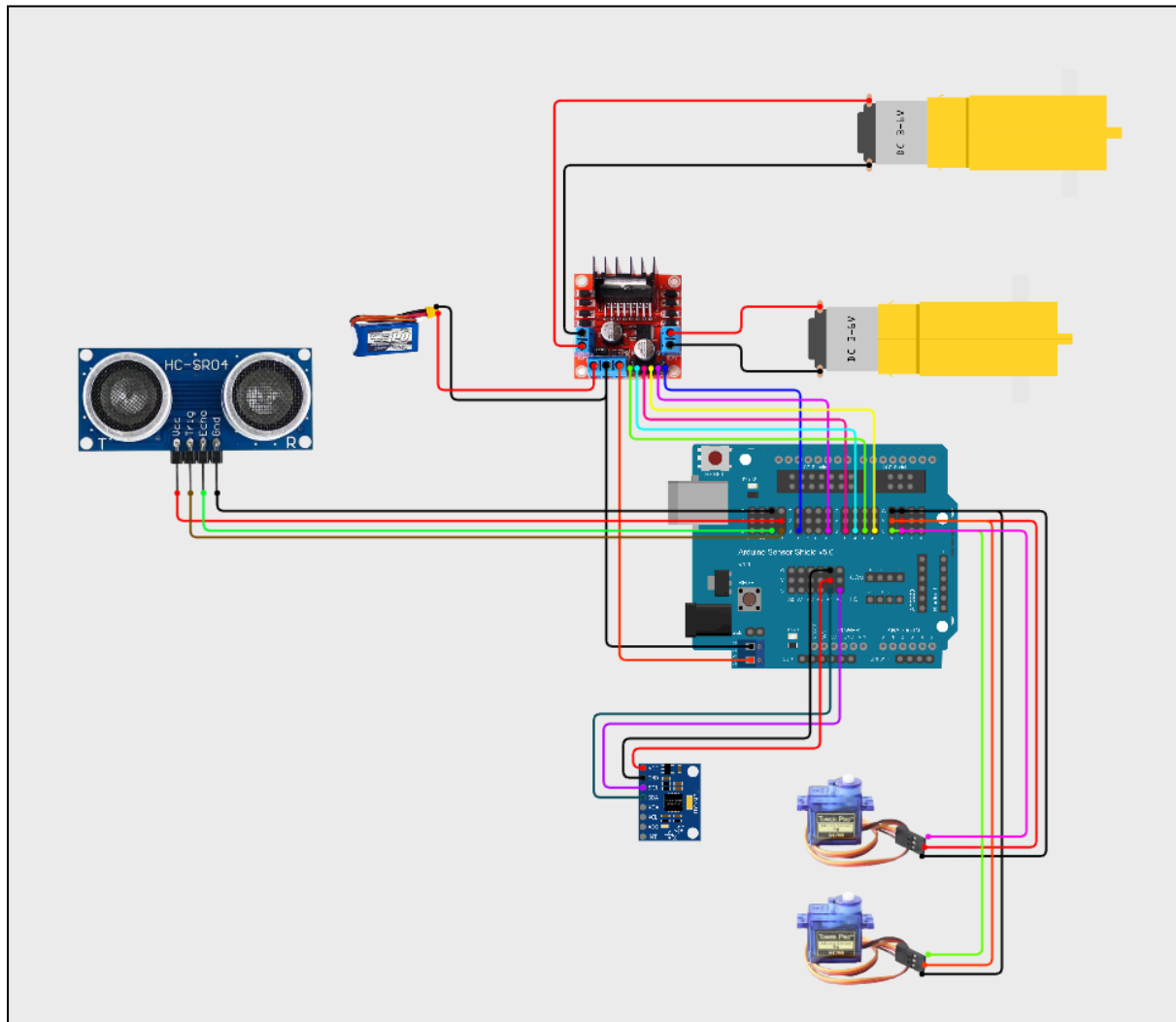
1.3 Final design



PHOTO

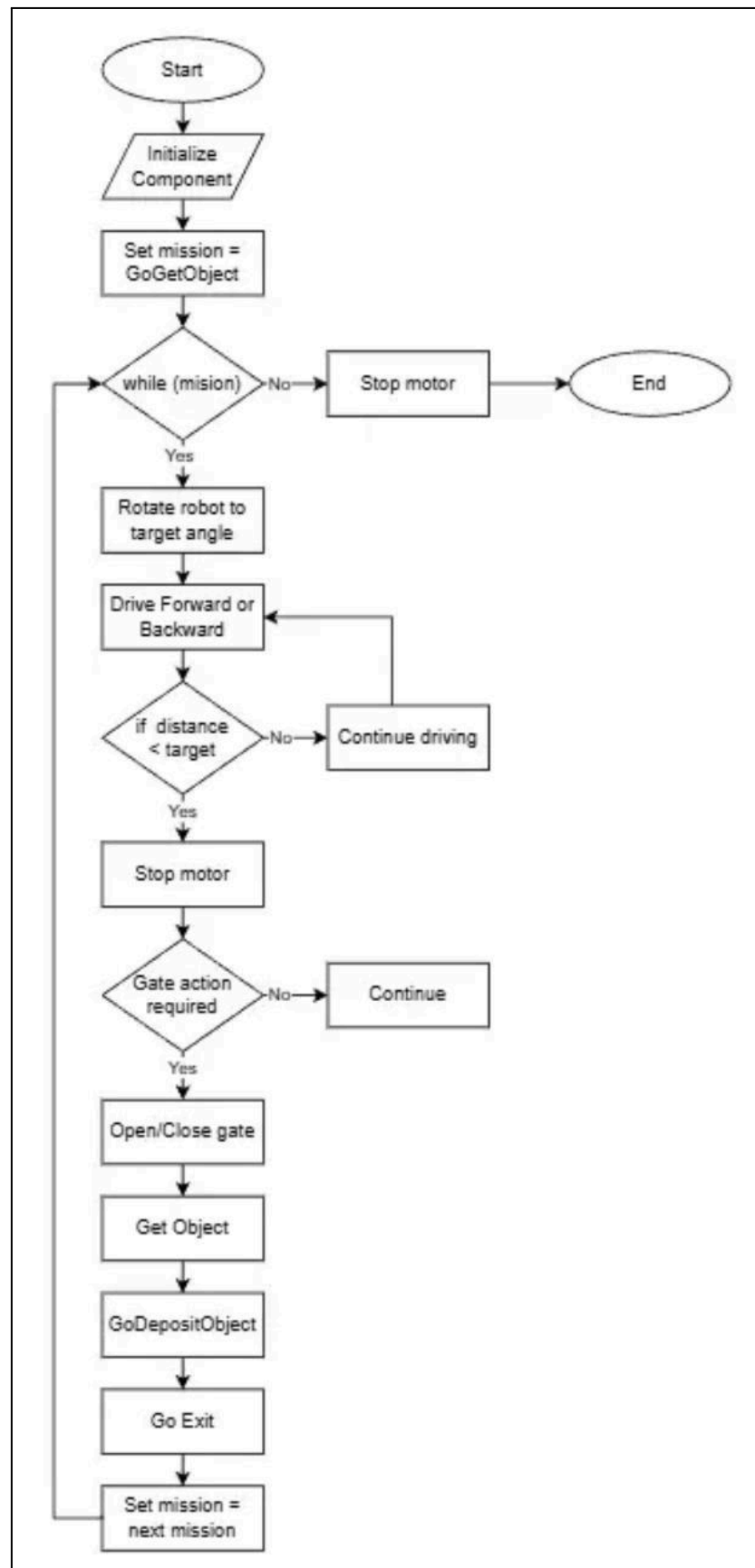


ELECTRICAL DIAGRAM



CODING

- FLOWCHART



- FULL CODE AND EXPLANATION

```
1  #include <Wire.h>
2  #include <Servo.h>
3  #include <MPU6050_tockn.h>
4
5  struct Step {
6      float angle;
7      float distance;
8      bool openGate;
9      bool closeGate;
10     bool backward;
11 };
12
13 MPU6050 mpu(Wire);
14 Servo servoLeft, servoRight;
15
16 #define MOTOR_A_EN 10
17 #define MOTOR_A_1 9
18 #define MOTOR_A_2 8
19 #define MOTOR_B_EN 5
20 #define MOTOR_B_1 6
21 #define MOTOR_B_2 7
22
23 #define TRIG_PIN 12
24 #define ECHO_PIN 13
25
26 int baseSpeed = 160;
27 float baseYaw = 0;
28 float kp = 0.8;
29
30 void motorControl(int leftSpeed, int rightSpeed);
31 void stopMotors();
32 void rotateTo(float targetAngle);
33 float readUltrasonic();
34 void driveUntil(float distance, bool backward);
35 void openGate();
36 void closeGate();
```

This part of the code sets up the fundamental structure, hardware definitions, and function declarations for an Arduino robot project that uses sensors and servos for motion and control. It begins by including three essential libraries: **Wire.h** for I2C communication, **Servo.h** for controlling servo motors, and **MPU6050_tockn.h** for reading data from the MPU6050 gyro sensor. A custom structure named **Step** is then defined to store parameters for each robot movement, including turning **angle**, travel **distance**, and Boolean flags for actions such as **openGate**, **closeGate**, and **backward**. Next, objects for the **MPU6050** sensor (**mpu**) and two servo motors (**servoLeft** and **servoRight**) are created. The motor pins for both left and right motors (**MOTOR_A** and **MOTOR_B**) are defined, along with the **TRIG** and **ECHO** pins used by the ultrasonic sensor for distance measurement. Several control constants are also initialized: **baseSpeed** (the normal motor speed), **baseYaw** (the initial orientation angle), and **kp** (the proportional control gain for rotation). Finally, a set of **function prototypes** is declared for key robot operations such as controlling motor speed, stopping, rotating to a specific angle, reading distance, driving a set distance, and opening or closing the servo-controlled gate which will be implemented later in the code.

```

37
38 Step missionGet[] = {
39     {0, 5, false, false, false},
40     {-90, 25, false, false, false},
41     {0, 45, false, false, false},
42     {90, 0, true, false, false},
43     {0, 5, false, true, false}
44 };
45
46 Step missionDrop[] = {
47     {180, 5, false, false, false},
48     {-90, 5, false, false, false},
49     {90, 5, false, false, false},
50     {90, 5, false, false, false},
51     {-90, 5, false, false, false},
52     {-90, 25, false, false, false},
53     {90, 5, false, false, false},
54     {-90, 5, false, false, false},
55     {-90, 5, false, false, false},
56     {90, 5, false, false, false},
57     {90, 5, true, false, false},
58     {0, 25, false, true, true}
59 };
60
61 Step missionExit[] = {
62     {90, 5, false, false, false},
63     {-90, 5, false, false, false},
64     {90, 5, false, false, false},
65     {90, 5, false, false, false},
66     {-90, 5, false, false, false},
67     {90, 5, false, false, false},
68     {90, 5, false, false, false},
69     {-90, 25, false, false, false},
70     {-90, 5, false, false, false},
71     {90, 5, false, false, false}
72 };

```

This section of the code defines three arrays — **missionGet**, **missionDrop**, and **missionExit** — each representing a different sequence of movement and action steps for the robot to perform during its operation. Each array is composed of elements based on the previously defined **Step** structure, where each step includes five parameters: the **rotation angle**, **travel distance**, and three Boolean flags (**openGate**, **closeGate**, **backward**) that specify additional actions.

missionGet describes the robot's path for approaching and collecting an object. It includes commands to move forward, turn at specific angles (e.g., -90° or 90°), open the servo gate to collect the object, and then close it after pickup.

missionDrop outlines the robot's path to transport and release the object. It involves multiple turns and straight movements to navigate toward the drop-off location, execute gate-opening actions to release the item, and finally close the gate while possibly reversing.

missionExit defines the route for the robot to leave the drop zone and return or exit the mission area, using a sequence of turns and short distance movements similar to the previous missions but without opening or closing the gate.

Overall, these structured arrays act as **predefined mission plans**, allowing the robot to follow step-by-step instructions autonomously without hardcoding movement logic inside the main loop.

```
73
74 void setup() {
75     Serial.begin(115200);
76     Wire.begin();
77     mpu.begin();
78     mpu.calcGyroOffsets(true);
79
80     servoLeft.attach(2);
81     servoRight.attach(3);
82
83     pinMode(MOTOR_A_EN, OUTPUT);
84     pinMode(MOTOR_A_1, OUTPUT);
85     pinMode(MOTOR_A_2, OUTPUT);
86     pinMode(MOTOR_B_EN, OUTPUT);
87     pinMode(MOTOR_B_1, OUTPUT);
88     pinMode(MOTOR_B_2, OUTPUT);
89     pinMode(TRIG_PIN, OUTPUT);
90     pinMode(ECHO_PIN, INPUT);
91
92     mpu.update();
93     baseYaw = mpu.getAngleZ();
94 }
95
96 void loop()
97     executeMission(missionGet, sizeof(missionGet) / sizeof(missionGet[0]));
98
99     executeMission(missionDrop, sizeof(missionDrop) / sizeof(missionDrop[0]));
100
101     executeMission(missionExit, sizeof(missionExit) / sizeof(missionExit[0]));
102
103     stopMotors();
104     while (true);
105 }
```

This part of the code initializes and defines the robot's main operational sequence. The **setup()** function is run once to prepare the hardware: it starts **serial communication** and **I2C communication** for sensors, initializes the **MPU-6050 sensor** (IMU) and calibrates its gyroscope, attaches the **servo motors** for mechanical actions, and sets the input/output roles for the **DC motor control** and **ultrasonic sensor** pins. It concludes by reading the initial heading from the MPU, storing it as **baseYaw** for navigation reference. The **loop()** function then runs the robot's complete mission by sequentially executing three defined phases: **missionGet**, **missionDrop**, and **missionExit**. After completing all three missions, it calls **stopMotors()** to disable movement and enters an **infinite while(true) loop**, effectively ending the robot's active program execution.

```

106
107 void executeMission(Step* missionArray, int size) {
108     for (int i = 0; i < size; i++) {
109         Step* stepPtr = &missionArray[i];
110
111         if (stepPtr->angle != 0) rotateTo(stepPtr->angle);
112         if (stepPtr->distance > 0) driveUntil(stepPtr->distance, stepPtr->backward);
113         if (stepPtr->openGate) openGate();
114         if (stepPtr->closeGate) closeGate();
115     }
116 }
117
118 void motorControl(int leftSpeed, int rightSpeed) {
119     leftSpeed = constrain(leftSpeed, -255, 255);
120     rightSpeed = constrain(rightSpeed, -255, 255);
121
122     if (leftSpeed >= 0) {
123         digitalWrite(MOTOR_A_1, HIGH);
124         digitalWrite(MOTOR_A_2, LOW);
125     } else {
126         digitalWrite(MOTOR_A_1, LOW);
127         digitalWrite(MOTOR_A_2, HIGH);
128     }
129     analogWrite(MOTOR_A_EN, abs(leftSpeed));
130
131     if (rightSpeed >= 0) {
132         digitalWrite(MOTOR_B_1, HIGH);
133         digitalWrite(MOTOR_B_2, LOW);
134     } else {
135         digitalWrite(MOTOR_B_1, LOW);
136         digitalWrite(MOTOR_B_2, HIGH);
137     }
138     analogWrite(MOTOR_B_EN, abs(rightSpeed));
139 }
140
141 void stopMotors() {
142     analogWrite(MOTOR_A_EN, 0);
143     analogWrite(MOTOR_B_EN, 0);
144 }

```

This segment of code contains the core **motion and mission control logic** for the robot. The **executeMission()** function is responsible for iterating through an array of mission steps and calling the appropriate action functions based on the step's data (angle, distance, openGate, or closeGate). The **motorControl()** function manages the **speed and direction of the drive motors**: it first ensures the input **leftSpeed** and **rightSpeed** values are within the acceptable range (-255 to 255) using **constrain()**; it then uses **digitalWrite()** on pins MOTOR_A_1/2 and MOTOR_B_1/2 to set the **direction**(forward or backward) and employs **analogWrite()** on the enable pins (MOTOR_A_EN and MOTOR_B_EN) with the absolute speed value to set the motor's **velocity** using **PWM**. Finally, the simple **stopMotors()** function immediately halts both motors by setting the PWM signal on their enable pins to 0.

```

118 void motorControl(int leftSpeed, int rightSpeed) {
140
141 void stopMotors() {
142     analogWrite(MOTOR_A_EN, 0);
143     analogWrite(MOTOR_B_EN, 0);
144 }
145
146 void rotateTo(float targetAngle) {
147     delay(500);
148     baseYaw = targetAngle;
149 }
150
151 void driveUntil(float distance, bool backward) {
152     int speed = backward ? -baseSpeed : baseSpeed;
153     motorControl(speed, speed);
154     while (readUltrasonic() > distance) {
155         delay(50);
156     }
157     stopMotors();
158 }
159
160 float readUltrasonic() {
161     digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
162     digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
163     digitalWrite(TRIG_PIN, LOW);
164     long duration = pulseIn(ECHO_PIN, HIGH);
165     return duration * 0.034 / 2;
166 }
167
168 void openGate() {
169     servoLeft.write(180);
170     servoRight.write(0);
171     delay(1500);
172 }
173
174 void closeGate() {
175     servoLeft.write(0);
176     servoRight.write(180);
177     delay(1500);
178 }

```

This section contains the specialized **movement and sensing functions** of the robot. The **readUltrasonic()** function measures distance by sending a brief pulse from the **TRIG_PIN**, waiting for the return echo on the **ECHO_PIN**, and converting the sound wave's travel time into centimeters. The **driveUntil()** function moves the robot forward or backward at a defined base speed using **motorControl()** and continuously checks the distance from the **ultrasonic sensor**; it stops the motors once the remaining distance is less than the target distance. The **rotateTo()** function simply sets a new **baseYaw** reference angle for navigation after a short pause. Lastly, the **openGate()** and **closeGate()** functions control the robot's mechanical action by commanding the two **servo motors** to move to their respective extreme angles (0 or 180 degrees) and pausing briefly to allow the action to complete.

CODE Explanation

The objective of this project is to design and implement an Arduino-controlled robot that can **autonomously navigate a predefined path**, perform **object collection and drop-off tasks**, and finally **return to the exit**.

The robot uses an **ultrasonic distance sensor** to detect obstacles, **servo motors** to operate a gate mechanism, and **DC motors** to move and rotate according to programmed instructions. At first we were planning on using gyro (MPU6050) but the coding part doesn't work so we use other methods to code without gyro instead.

- **Functions** : for organized, reusable code blocks.
- **Structure (struct)** : to store mission step data such as angle, distance, and gate action.
- **Array** : to hold multiple mission steps for each phase of the route.
- **Pointer** : to iterate and access mission step data dynamically.
- **Loops** : to repeatedly execute mission sequences automatically.

Explanation of Main Components

A structure named **MissionStep** is declared to store

- **targetAngle** : defines how much the robot should rotate before moving forward
- **distance** : defines the forward movement distance in centimeters measured by the ultrasonic sensor.
- **gateAction** : determines whether to open or close the servo gate during that step.

This makes it easy to represent every movement and action as one simple, structured dataobject.

Array and Pointer

The program defines **three separate arrays** to represent the route phases:

1. **getObject[]** → path for collecting the object
2. **dropObject[]** → path for delivering the object
3. **exitPath[]** → path to return to the exit

Each array contains multiple **MissionStep** elements describing angles, distances, and gate actions.

A pointer `MissionStep* missionPtr` is used to access the array dynamically during loop execution:

```
107 void executeMission(Step* missionArray, int size) {
108     for (int i = 0; i < size; i++) {
109         Step* stepPtr = &missionArray[i];
110
111         if (stepPtr->angle != 0) rotateTo(stepPtr->angle);
112         if (stepPtr->distance > 0) driveUntil(stepPtr->distance, stepPtr->backward);
113         if (stepPtr->openGate) openGate();
114         if (stepPtr->closeGate) closeGate();
115     }
116 }
```

This **loop** executes:

Rotation : robot turns to target angle.

Forward movement : continues until ultrasonic sensor reads the specified distance.

Gate control : opens/closes if `gateAction = true`.

Using `for` loops reduces code repetition and enables multiple mission sequences to run automatically.

Functions

`setupMotors()` : Initializes motor driver pins.

`driveStraight(int speed)` : Moves robot forward at given speed.

`stopMotors()` : Stops both motors.

`readUltrasonic()` : Reads distance in cm using the ultrasonic sensor.

`rotateTo(float targetAngle)`: Simulates rotation to a desired angle.

`openGate()` / `closeGate()`: Controls servo motors for gate mechanism.

Control Flow Explanation (Logic)

Go Get Object

- Robot starts moving forward until the ultrasonic sensor detects an object at **5 cm**.
- Turns **left 90°** and continues **25 cm** forward.
- Turns **right 90°**, moves **45 cm**, then **right 90°** again.
- At this position, the robot **opens the gate** to collect the object.
- Moves forward **5 cm**, then **closes the gate**.

Go Drop Object

- Robot **turns 180°**, retraces its path through a sequence of **left and right turns** and forward moves (5–25 cm each).
- After reaching the drop location, the robot **opens the gate** to release the object.
- Moves **backward 25 cm**, then **closes the gate** to prepare for exit.

Go to EXIT

- The robot follows a complex navigation path using multiple 90° turns and 5 cm forward moves.
- Finally stops when it reaches the **final exit point**, completing the mission.
- Each phase is represented as a **series of MissionSteps** processed in sequence by the same loop logic — demonstrating modular programming and structural reusability.

Sensor and Actuator Coordination

The **ultrasonic sensor** (using **TRIG** and **ECHO** pins) constantly measures distance using sound wave reflection.

It controls when the robot should stop moving forward.

The **servo motors** attached to the gate act as the mechanical arm for grabbing or releasing the object.

DC motors are used for driving and turning, controlled by PWM signals (**analogWrite()**).

MANUAL ROBOT

COMPONENT LIST

- 180 Optical Encoder Motor

Parameter	Specification
Rated voltage	7.4 V
Gear reduction ratio	39.6 : 1
No-load current	240 mA
Load current	≤ 750 mA
No-load speed	$350 \pm 5\%$ RPM
Original motor speed	$14,000 \pm 5\%$ RPM
Start torque	5 kg cm
Rated torque	0.8 kg cm
Output shaft length	9 mm
Power	3.7 W
Encoder rotation angle	360°
Motor type	DC motor with encoder
Control method	PWM or closed-loop feedback via encoder

- MS-1.5A Servo

Parameter	4.6 V	6.0 V	Unit	Tolerance
Running speed (no load)	0.11 s/60°	0.10 s/60°	s/60°	± 0.01 s
Operating current (no load)	270 mA	380 mA	mA	± 50 mA
Stall torque (locked)	1.5 kg·cm	1.8 kg·cm	kg·cm	± 0.30 kg·cm
Stall current (locked)	1200 mA	1500 mA	mA	± 50 –70 mA
Idle current (when stopped)	4 mA	4 mA	mA	± 1 mA
Service life (no load)	>180,000 cycles	>150,000 cycles	cycles	–

- Digital Servo MEDS15

Parameter	Specification
Operating voltage range	4.8 V to 6.8 V
No-load idle current	4-5 mA
No-load speed	62 - 71 RPM (at 6.8V)
Stall torque	18 - 21.5 kg cm (at 6.8V)
Stall current	1.5 - 2.2 A (at 5 - 6.8V)
Control signal	PWM (pulse-width modulated)
Rotation Range	180°

- TT DC Motor Single-Shaft

Parameter	Specification
Gear Reduction Ratio	1:48
Rated Voltage	3 – 6 V
No-load Speed	≈200 RPM at 6V
No-load Current	≈150 mA at 6V
Stall Torque	≈0.8 kg cm at 6V
Stall Current	≈700 mA
Shaft Type	Single D-shaped output shaft
Gearbox Material	Plastic
Dimensions	≈70 × 22 × 18 mm
Weight	≈30 – 50 g
Power	≈0.9 W

- mBot2 Shield

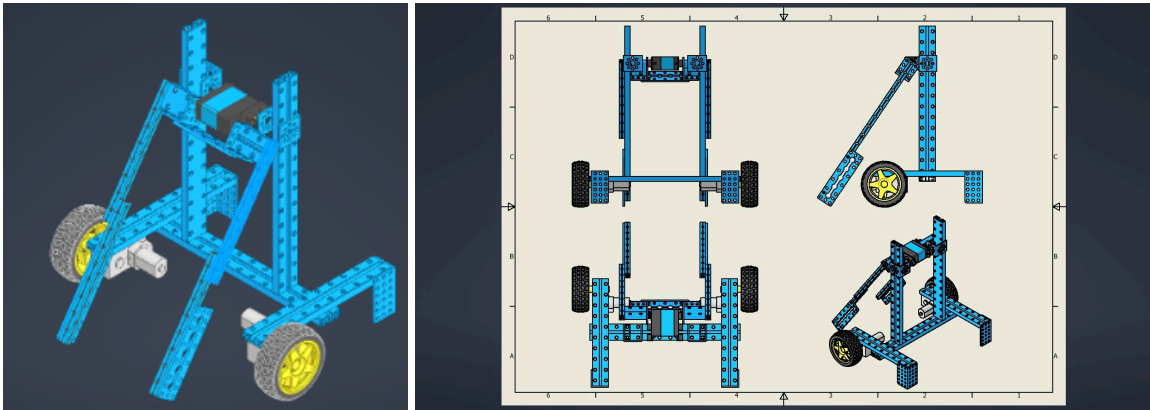
Component	Specification
Main Controller	CyberPi
Processor	ESP32-WROVER-B Dual-Core
Memory	8 MB Flash, 8 MB PSRAM
Display	1.44-inch Full-Color TFT
Communication	Wi-Fi 2.4GHz / Bluetooth 5.0
Interface Ports	1×USB-C, 1×mBuild Port, 1×Extension Port
Power Supply	3.7V Li-ion Battery (2500 mAh typical)
Operating Voltage	3.3V Logic Level
Programming	mBlock 5 / Python / Arduino Mode

- Bluetooth Controller

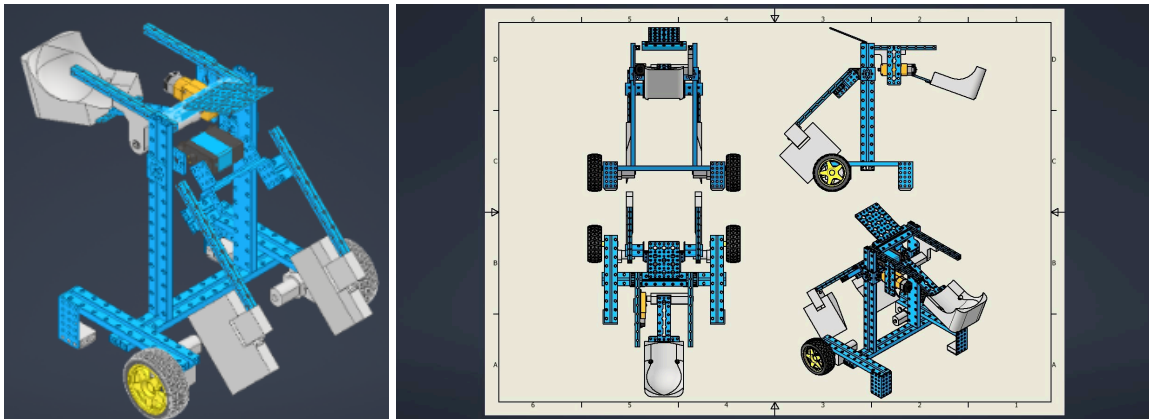
Parameter	Specification
Material	ABS
Bluetooth Version	4.0
Trasmission Distance	20 m
Power supply	5AA batteries

ASSEMBLY DESIGN

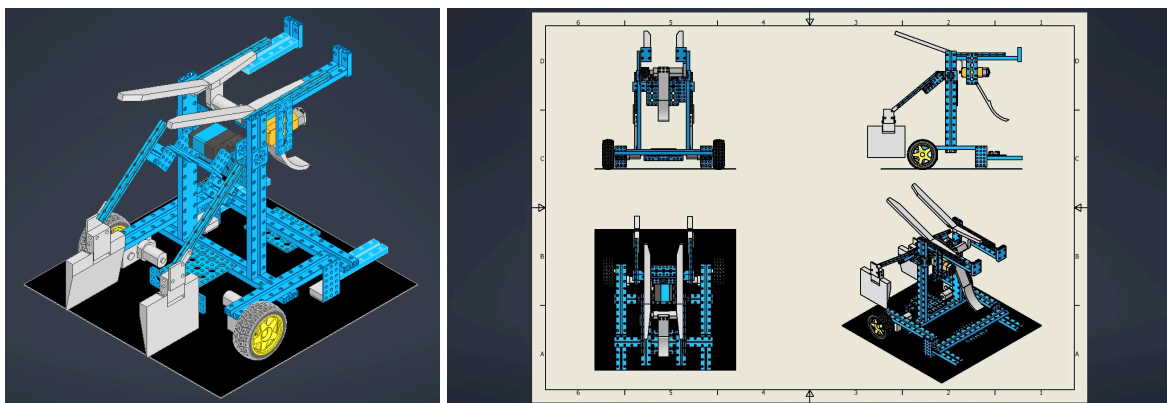
1.1 Version 1



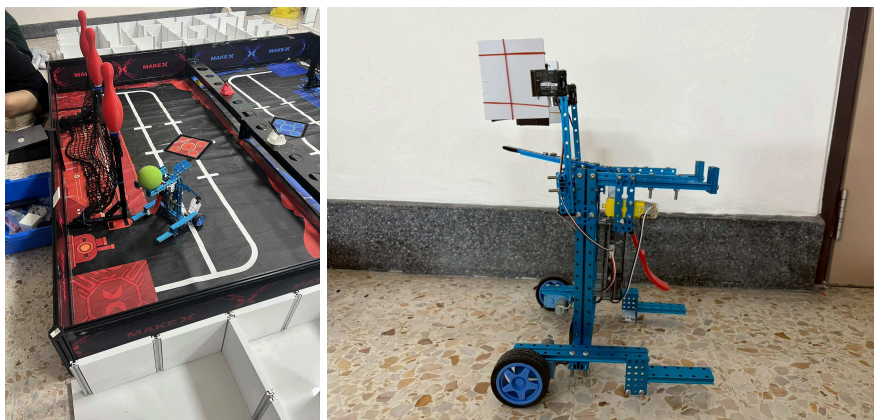
1.2 Version 2



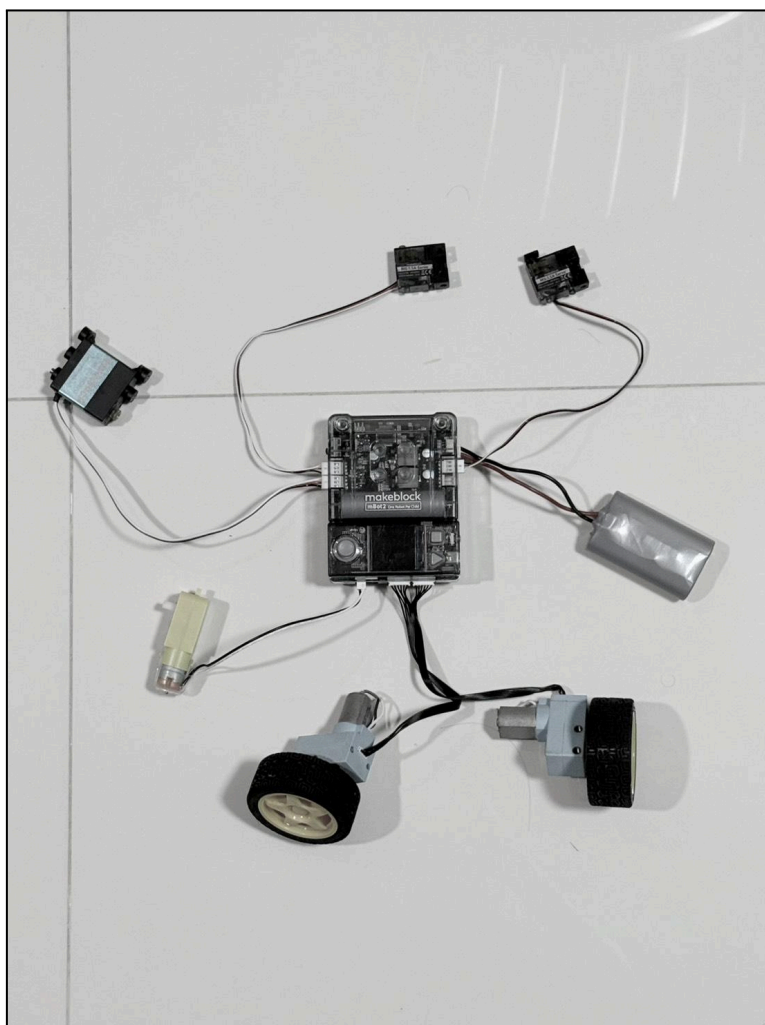
1.3 Final Design



PHOTO

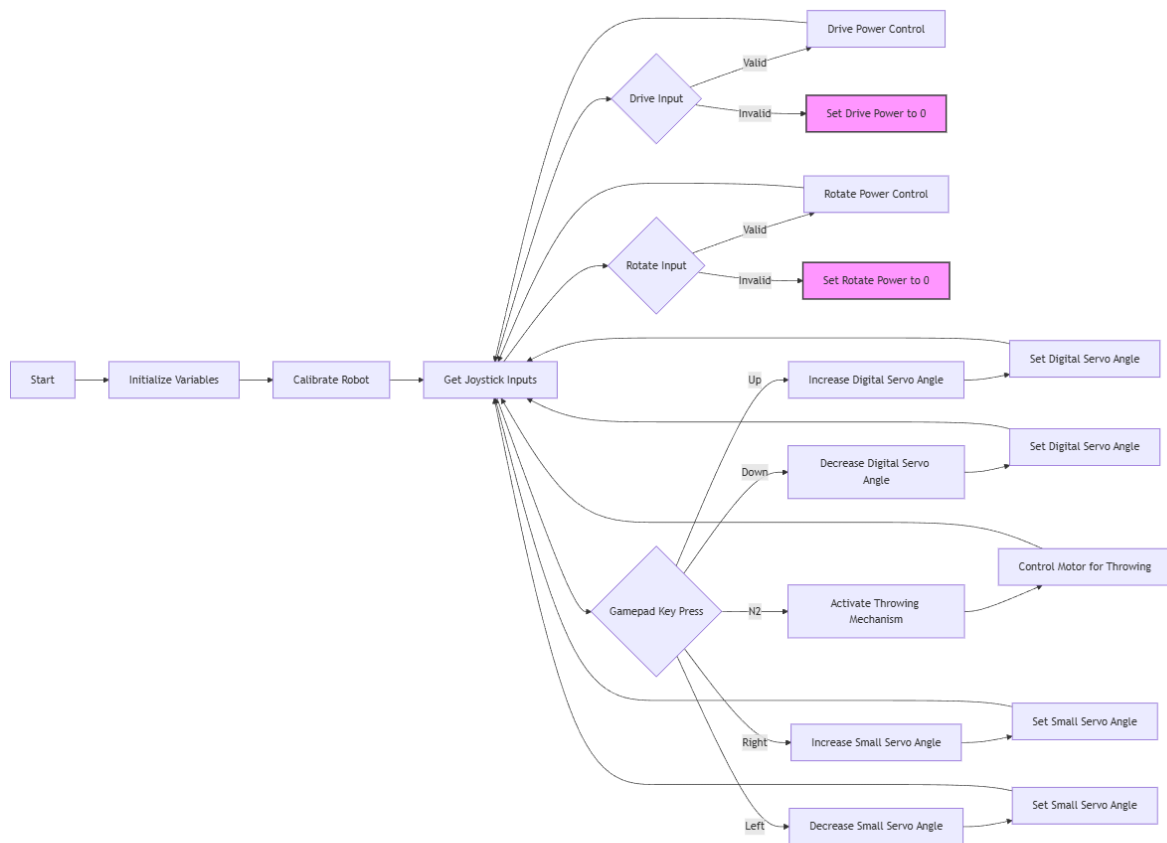


ELECTRICAL DIAGRAM



CODING

- FLOWCHART



- FULL CODE AND EXPLANATION

- Setup required libraries
- Creating constant variables for a better debugging process.

```
1  import gamepad
2  import time
3  from cyberpi import *
4
5  console.println('start')
6
7  ## Constant Variable
8  digital_servo_angle = 20
9  small_servo_angle = 0
10 drive_power_ratio = 0.8 # 2 <--> 0
11 angle_limit_up = 160
12 angle_limit_down = 20
13 angle_ratio = 2.5 # A multiple of 5 only (5, 10, 15 ...)
14 small_angle_limit_up = 90
15 small_angle_limit_down = 0
16 small_angle_ratio = 2.5 # A multiple of 5 only (5, 10, 15 ...)
17 launched = 0
18 throwing_speed = 200
19 pulling_back_speed = 60
20
```

- Sets initial servo position when started.

```
21 ## Calibration
22 mbot2.servo_set(angle_limit_down, 's2')
23 mbot2.servo_set(small_servo_angle, 's3')
24 mbot2.servo_set(180-small_servo_angle, 's1')
25
26 console.println("Robot Calibrated")
27
```

- Begin a main loop
- Get live variables from the controller. (joystick position)

```
28 while True:
29     ### Testing
30     # mbot2.motor_set(-200, 'M1')
31     # time.sleep(0.3)
32     # mbot2.motor_set(0, 'M1')
33     # break
34
35
36     ### Get live variable
37     jstick_ry = gamepad.get_joystick('Ry')
38     jstick_rx = gamepad.get_joystick('Rx')
39     jstick_ly = gamepad.get_joystick('Ly')
40     jstick_lx = gamepad.get_joystick('Lx')
41
42     # console.println(jstick_rx)
43     # console.println(jstick_ly)
44     # console.println("-----")
45     # time.sleep(0.01)
46
```

- Get the position of the joystick and check whether it moves or not.
- If the position is over 5, the encoder will move with the power be a fraction of the constant ratio to the position of the joystick.

```

47     ### Encoder function
48     drive = jstick_ly
49     rotate = jstick_rx
50
51     if drive >= 5 or drive <= -5:
52         mbot2.drive_power(-drive * drive_power_ratio, drive * drive_power_ratio)
53
54     if rotate >= 5 or rotate <= -5:
55         mbot2.drive_power(rotate * drive_power_ratio, rotate * drive_power_ratio)
56
57     if (drive <= 5 and drive >= -5) and (rotate <= 5 and rotate >= -5):
58         mbot2.drive_power(0, 0)
59
60
61

```

- When the up or down key is pressed, the servo angle increases or decreases according to the angle ratio.
- If the angle is over the set limit, it will stop adding or subtracting the servo angle for safety reasons.

```

62     ### Collecting mechanism
63     if gamepad.is_key_pressed('Up'):
64         if digital_servo_angle < angle_limit_up:
65             digital_servo_angle += angle_ratio
66         else:
67             console.print('Servo exceed up limit of ')
68             console.println(angle_limit_up)
69             mbot2.servo_set(digital_servo_angle, 's2')
70             time.sleep(0.01)
71     if gamepad.is_key_pressed('Down'):
72         if digital_servo_angle > angle_limit_down:
73             digital_servo_angle -= angle_ratio
74         else:
75             console.print('Servo exceed down limit of ')
76             console.println(angle_limit_down)
77             mbot2.servo_set(digital_servo_angle, 's2')
78             time.sleep(0.01)
79
80

```

- If the N2 button is pressed, the dc motor speed is set to run for a specific time and stop.
- Then, it moves back to the original position using the same method.

```

81     ### DC motor
82     if gamepad.is_key_pressed('N2'):
83         mbot2.motor_set(-throwing_speed, 'M1')
84         time.sleep(0.175)
85         mbot2.motor_set(0, 'M1')
86         mbot2.motor_set(pulling_back_speed, 'M1')
87         time.sleep(0.3)
88         mbot2.motor_set(0, 'M1')
89
90

```

- The code links two small servos together so that it was able to grab the ball using the same method from the collecting mechanism.

```

91     ### Grabbing mechanism
92     if gamepad.is_key_pressed('Right'):
93         if small_servo_angle < small_angle_limit_up:
94             small_servo_angle += small_angle_ratio
95         else:
96             console.print('Servo exceed up limit of ')
97             console.println(small_angle_limit_up)
98             console.println("Servo is fully closed")
99
100         mbot2.servo_set(small_servo_angle, 's3')
101         mbot2.servo_set(180-small_servo_angle, 's1')
102         time.sleep(0.01)
103
104     if gamepad.is_key_pressed('Left'):
105         if small_servo_angle > small_angle_limit_down:
106             small_servo_angle -= small_angle_ratio
107         else:
108             console.print('Servo exceed down limit of ')
109             console.println(small_angle_limit_down)
110             console.println("Servo is fully opened")
111
112         mbot2.servo_set(small_servo_angle, 's3')
113         mbot2.servo_set(180-small_servo_angle, 's1')
114         time.sleep(0.01)
115

```

Group Evaluation

NAME	CONTRIBUTION	EVALUATED BY	CONFIRM BY
Nathi Ritnatikul	100%	ผู้ช่วย ฤทธิเดช	ผู้ช่วย ฤทธิเดช
Kullatida Raksanaves	100%	ผู้ช่วย ฤทธิเดช	Kullatida Raksanaves
Thannapat Ketbunchoo	100%	ผู้ช่วย ฤทธิเดช	